

Khant Thu Aung/ 105292912

11.1P (Clock in Another Language)

Counter.py

```
class Counter:
```

```
    def __init__(self, name):
```

```
        self._name = name
```

```
        self._count = 0
```

```
    def increment(self):
```

```
        self._count += 1
```

```
    def reset(self):
```

```
        self._count = 0
```

```
    @property
```

```
    def name(self):
```

```
        return self._name
```

```
    @name.setter
```

```
    def name(self, value):
```

```
        self._name = value
```

```
    @property
```

```
    def ticks(self):
```

```
        return self._count
```

```
def reset_by_default(self):  
    try:  
        large_value = 2147483647912  
        if large_value > 2147483647:  
            self._count = ((large_value - 2147483648) % (2**32)) - 2147483648  
        else:  
            self._count = large_value  
    except:  
        # Handle any potential issues  
        self._count = 0
```

Clock.py

```
from counter import Counter
```

```
class Clock:
```

```
    def __init__(self):  
        self._hour = Counter("Hour")  
        self._minute = Counter("Minute")  
        self._second = Counter("Second")
```

```
    def tick(self):  
        self._increment_second()
```

```
    def _increment_second(self):  
        self._second.increment()  
        if self._second.ticks == 60:  
            self._second.reset()
```

```
        self._increment_minute()

def _increment_minute(self):
    self._minute.increment()
    if self._minute.ticks == 60:
        self._minute.reset()
        self._increment_hour()

def _increment_hour(self):
    self._hour.increment()
    if self._hour.ticks == 13:
        self._hour.reset()
        self._hour.increment()

def reset(self):
    self._hour.reset()
    self._minute.reset()
    self._second.reset()

def get_time(self):
    return f"{self._hour.ticks:02d}:{self._minute.ticks:02d}:{self._second.ticks:02d}"
```

Main.py

```
from clock import Clock

import os

import psutil
```

```
def main():  
    second_total = 86400  
    my_clock = Clock()  
  
    # Run for 3600 seconds (1 hour) first  
    for i in range(3600):  
        my_clock.tick()  
    # Then run for a full day (86400 seconds)  
    for i in range(second_total):  
        my_clock.tick()  
        print(my_clock.get_time())  
  
    # Get the current process  
    proc = psutil.Process(os.getpid())  
  
    print(f"Current process: {proc}")  
    # Display the total physical memory size allocated for the current process  
    print(f"Physical memory usage: {proc.memory_info().rss} bytes")  
    # Display peak physical memory usage (only available on some platforms)  
    try:  
        print(f"Peak physical memory usage: {proc.memory_info().peak_wset} bytes")  
    except AttributeError:  
        print("Peak physical memory usage not supported on this platform.")
```

```
if __name__ == "__main__":  
    main()
```

Comparison

C#

```
12:59:59  
01:00:00  
Current process: System.Diagnostics.Process (Clock)  
Physical memory usage: 41844736 bytes  
Peak physical memory usage 0 bytes  
~/De/A/OOPAssignments/P31/Clock main !122 ?6 >
```

Python

B